



PUC Minas

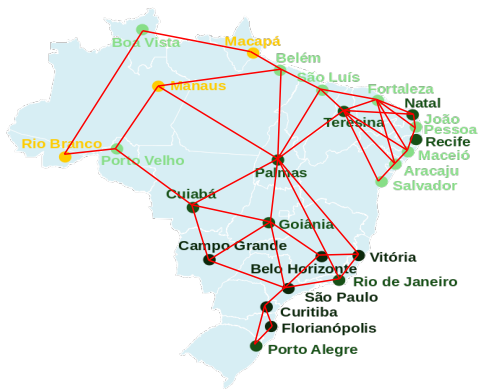
Teoria dos Grafos e Computabilidade

Problema do caminho mínimo

Prof. Gabriel B. da Fonseca
Email: gbfonseca@sga.pucminas.br

Problema do caminho mínimo - Introdução

- Uma pessoa deseja ir de Belo Horizonte até Cuiabá.
- Essa pessoa procura qual a melhor rota (a de menor distância) para chegar ao seu destino.
- Dado um mapa rodoviário do Brasil contendo as **possíveis conexões** entre cidades e as **distâncias dessas conexões**, como podemos **determinar a rota mais curta?**



- Dado um grafo ponderado $G = (V, E)$
- E uma função de pesos $w : E \rightarrow \mathbb{R}$
- **O peso/custo de um caminho** $p_{v_0, v_k} = \langle v_0, v_1, \dots, v_{k-1}, v_k \rangle$, definido como $w(p_{v_0, v_k})$, é dado pela **soma do peso todas as arestas** que o constituem:

$$w(p_{v_0, v_k}) = \sum_{i=1}^k w(v_{i-1}, v_i)$$

Definição

- Dado um grafo ponderado $G = (V, E)$
- E uma função de pesos $w : E \rightarrow \mathbb{R}$
- **O peso/custo de um caminho** $p_{v_0, v_k} = \langle v_0, v_1, \dots, v_{k-1}, v_k \rangle$, definido como $w(p_{v_0, v_k})$, é dado pela **soma do peso todas as arestas** que o constituem:

$$w(p_{v_0, v_k}) = \sum_{i=1}^k w(v_{i-1}, v_i)$$

Caminho mínimo

O caminho mínimo **entre um par de vértices** (x, y) é dado pelo caminho $p_{x,y}$ que possui o menor peso $w(p_{x,y})$ possível (dentre todos os caminhos existentes entre x e y).

- Caminhos mínimos de fonte/origem única
 - Caminho mínimo entre uma fonte e um único vértice (caminho mínimo entre um par de vértices)
 - Caminho mínimo de uma única fonte para todos os outros vértices
- Caminhos mínimos entre todos os pares

- Caminhos mínimos de fonte/origem única
 - Caminho mínimo entre uma fonte e um único vértice (caminho mínimo entre um par de vértices)
 - **Caminho mínimo de uma única fonte para todos os outros vértices**
- Caminhos mínimos entre todos os pares

- O algoritmo de Dijkstra resolve o problema de caminhos mais curtos **de uma única origem** em um grafo ponderado $G = (V, E)$
- Para o funcionamento correto, o grafo **não pode conter arestas de peso negativo**
- Em outras palavras, $w : E \rightarrow \mathbb{R}^+$

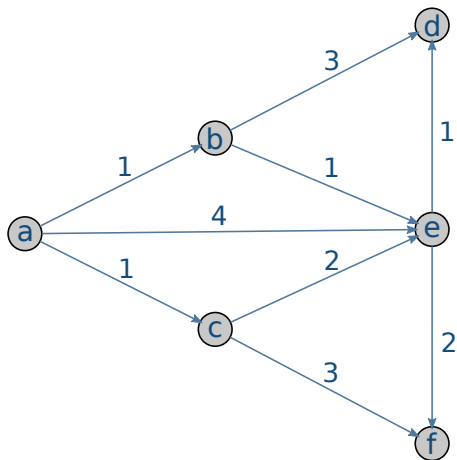
Dado um Grafo $G = (V, E)$, uma função de pesos w , e um vértice inicial (origem) s :

- Marcamos todos os vértices do grafo como **não visitados**.
- A cada iteração, começando por s até que todos os vértices sejam visitados:
 - Seleccionamos o vértice u **não visitado** que possui a menor **estimativa de caminho mais curto** a partir de s
 - Marcamos u como visitado
 - **Relaxa** todas as arestas $\{(u, x), x \in Adj[u]\}$
- Ao final, temos os **pesos dos caminhos mínimos** partindo de s até todos os outros vértices. Detalhe: **não temos os caminhos de fato!** Para isso calculamos também uma lista de predecessores.

Algorithm 1: Dijkstra(G, w, s)

```
1 for cada vértice  $v \in V$  do
2    $d[v] \leftarrow \infty$ 
3    $\pi[v] \leftarrow \text{NULL}$ 
4  $d[s] \leftarrow 0$ 
5  $S \leftarrow \emptyset$ 
6  $Q \leftarrow V$ 
7 while  $Q \neq \emptyset$  do
8    $u \leftarrow \text{ExtractMin}(Q)$ 
9    $S \leftarrow S \cup \{u\}$ 
10  for cada vértice  $v \in \text{Adj}[u]$  do
11    if  $d[v] > d[u] + w(u, v)$  then
12       $d[v] \leftarrow d[u] + w(u, v)$ 
13       $\pi[v] \leftarrow u$ 
```

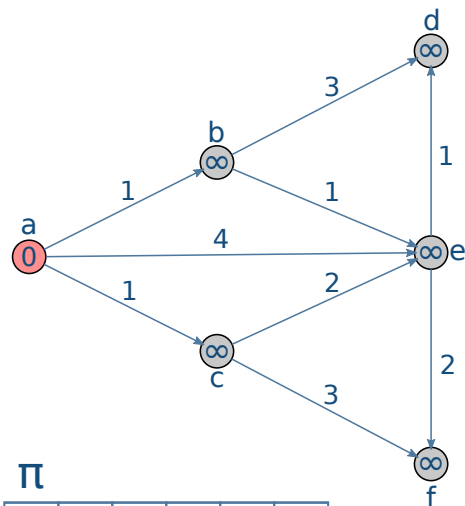
Dijkstra - Exemplo de execução



Q

a	
b	
c	
d	
e	
f	

Dijkstra - Exemplo de execução



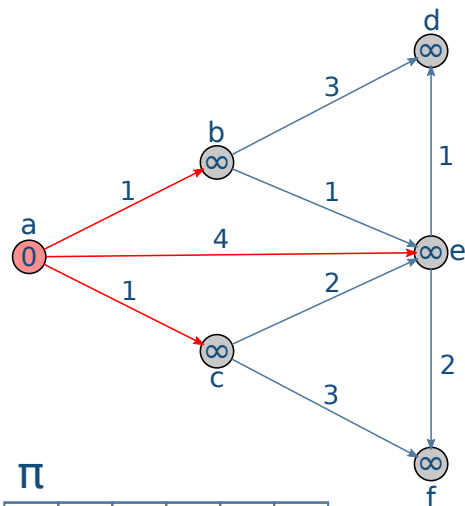
π

a	b	c	d	e	f

Q

a	0
b	∞
c	∞
d	∞
e	∞
f	∞

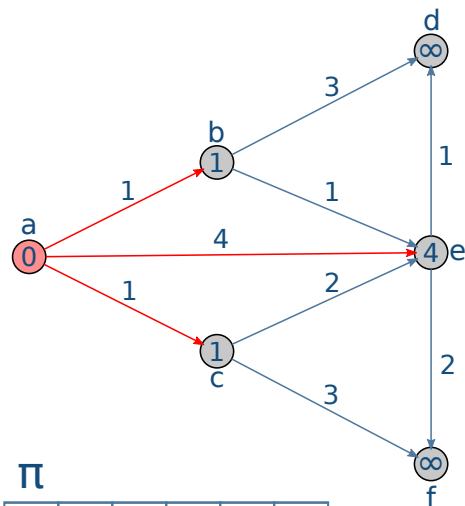
Dijkstra - Exemplo de execução



Q	
a	0
b	∞
c	∞
d	∞
e	∞
f	∞

π					
a	b	c	d	e	f

Dijkstra - Exemplo de execução



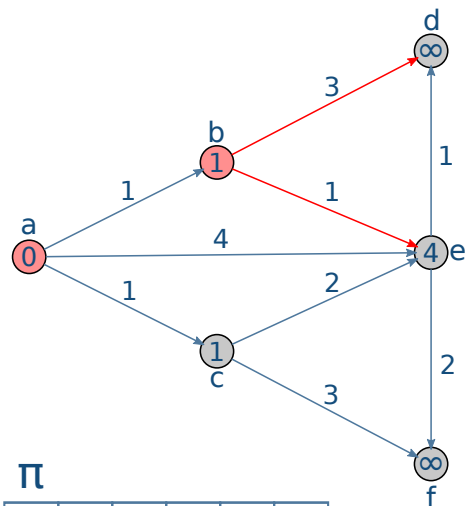
π

a	b	c	d	e	f
	a	a		a	

Q

a	0
b	1
c	1
d	∞
e	4
f	∞

Dijkstra - Exemplo de execução



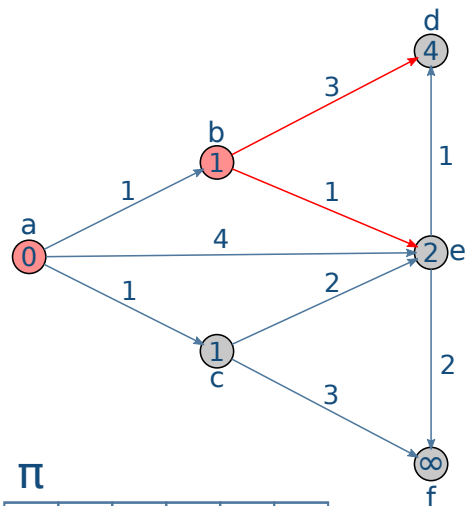
π

a	b	c	d	e	f
	a	a		a	

Q

a	0
b	1
c	1
d	∞
e	4
f	∞

Dijkstra - Exemplo de execução



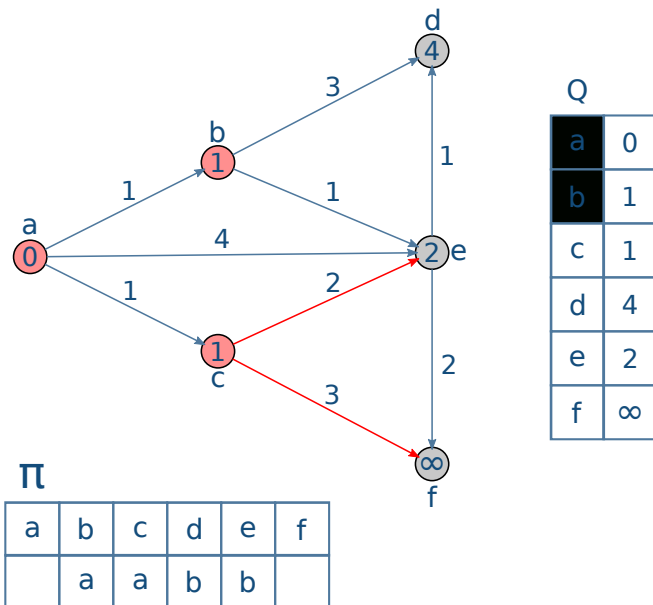
π

a	b	c	d	e	f
	a	a	b	b	

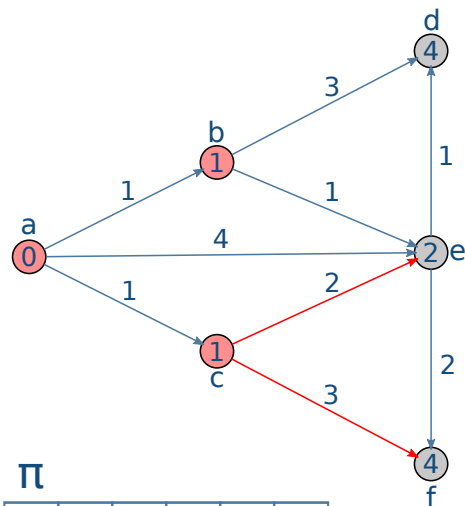
Q

a	0
b	1
c	1
d	4
e	2
f	∞

Dijkstra - Exemplo de execução



Dijkstra - Exemplo de execução



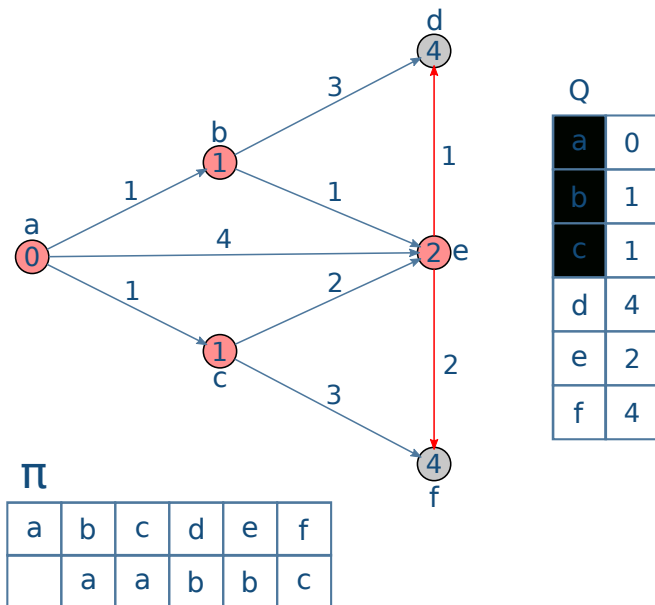
π

a	b	c	d	e	f
	a	a	b	b	c

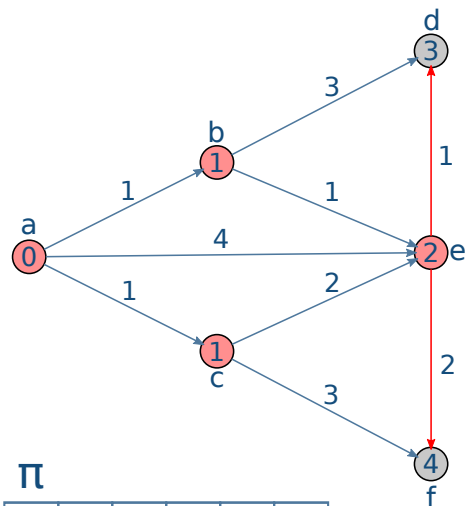
Q

a	0
b	1
c	1
d	4
e	2
f	4

Dijkstra - Exemplo de execução



Dijkstra - Exemplo de execução



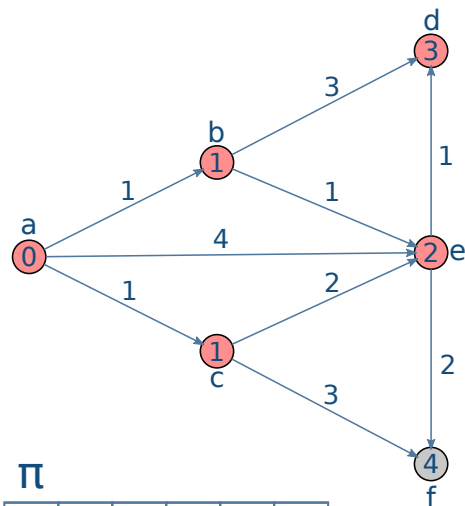
π

a	b	c	d	e	f
	a	a	e	b	c

Q

a	0
b	1
c	1
d	3
e	2
f	4

Dijkstra - Exemplo de execução



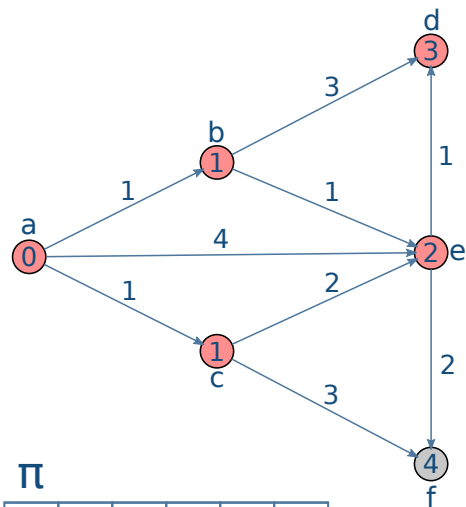
π

a	b	c	d	e	f
	a	a	e	b	c

Q

a	0
b	1
c	1
d	3
e	2
f	4

Dijkstra - Exemplo de execução



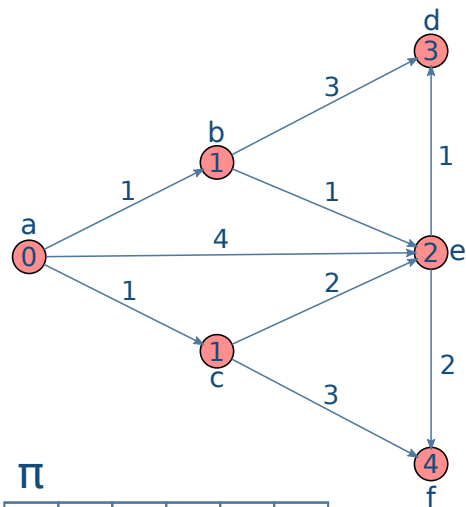
π

a	b	c	d	e	f
	a	a	e	b	c

Q

a	0
b	1
c	1
d	3
e	2
f	4

Dijkstra - Exemplo de execução



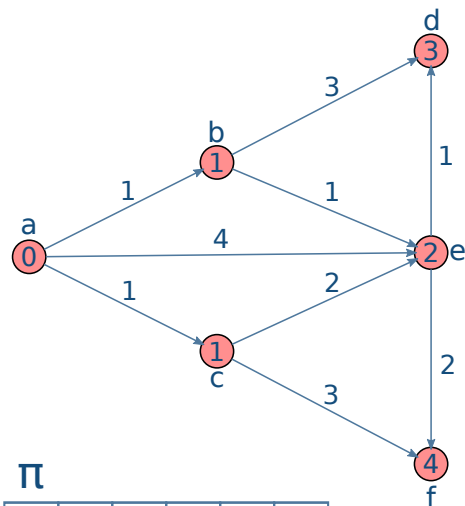
π

a	b	c	d	e	f
	a	a	e	b	c

Q

a	0
b	1
c	1
d	3
e	2
f	4

Dijkstra - Exemplo de execução



π

a	b	c	d	e	f
	a	a	e	b	c

Q

a	0
b	1
c	1
d	3
e	2
f	4

Algorithm 1: Dijkstra(G, w, s)

```
1 for cada vértice  $v \in V$  do }  $|V|$ 
2    $d[v] \leftarrow \infty$ 
3    $\pi[v] \leftarrow NULL$ 
4  $d[s] \leftarrow 0$ 
5  $S \leftarrow \emptyset$ 
6  $Q \leftarrow V$ 
7 while  $Q \neq \emptyset$  do
8    $u \leftarrow ExtractMin(Q)$  }  $|V|$ 
9    $S \leftarrow S \cup \{u\}$ 
10  for cada vértice  $v \in Adj[u]$  do }
11    if  $d[v] > d[u] + w(u, v)$  then }  $|E|$ 
12       $d[v] \leftarrow d[u] + w(u, v)$ 
13       $\pi[v] \leftarrow u$ 
```

$Q =$ Lista comum

- $ExtractMin(Q) = O(V)$
- Atualizar pesos = $O(1)$
- Complexidade total = $O(V) + O(V^2) + O(E) = O(V^2)$

Algorithm 1: Dijkstra(G, w, s)

```
1 for cada vértice  $v \in V$  do }  $|V|$ 
2    $d[v] \leftarrow \infty$ 
3    $\pi[v] \leftarrow NULL$ 
4  $d[s] \leftarrow 0$ 
5  $S \leftarrow \emptyset$ 
6  $Q \leftarrow V$   $\longrightarrow$  Build Min-heap =  $O(V \log(V))$ 
7 while  $Q \neq \emptyset$  do }  $|V|$ 
8    $u \leftarrow \text{ExtractMin}(Q)$ 
9    $S \leftarrow S \cup \{u\}$ 
10  for cada vértice  $v \in \text{Adj}[u]$  do }  $|E|$ 
11    if  $d[v] > d[u] + w(u, v)$  then
12       $d[v] \leftarrow d[u] + w(u, v)$ 
13       $\pi[v] \leftarrow u$ 
```

$Q =$ Fila de prioridade mínima (min-heap)

- $\text{ExtractMin}(Q) = O(\log V)$
- Atualizar pesos = $O(\log V)$
- Complexidade total = $O(V + V \log V) + O(V \log V) + O(|E| \log(V))$
= $O((V + E) \log V)$

Navegação por satélite (Google maps): Encontrar o caminho mais rápido entre RJ e SP

Redes complexas/Redes sociais: Sugestões de amizade para um usuário. O algoritmo padrão Dijkstra pode ser aplicado para descobrir o caminho mais curto entre usuários medido por meio ou conexões entre eles.

Redes de computadores: Protocolos de roteamento, utilizados para encontrar qual o melhor caminho entre um roteador fonte e um roteador destino.

Próxima aula:

- Continuação de caminhos mínimos
- Grafos com pesos negativos
- Bellman-Ford

Atividades

- Fazer lista de exercícios

Perguntas?